МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ 1

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФеП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

Варіант 11: Мобільний тариф

1. Мета та завдання роботи

- Зрозуміти принципи чистої функції та референтної прозорості.
- Навчитися виявляти та ізолювати побічні ефекти (I/O, мутації, глобальний стан).

  Застосувати параметризацію обчислень за допомогою typing.Callable та функцій вищого порядку.

- Реалізувати індивідуальний варіант №11 («Мобільний тариф»): обчислити місячну вартість використаних послуг (хвилини, SMS, інтернет-трафік) з урахуванням тарифних правил, бонусів та сформувати нове поле monthly_cost без мутації вхідних даних.

2. Опис і назви методів (функцій), реалізованих у проєкті

Згідно з вимогами лабораторної роботи, програмний код розділено на чисте функціональне ядро (core.py) та ізольовану оболонку побічних ефектів (app.py). Нижче наведено перелік і детальний опис усіх функцій,використаних у реалізації Варіанта 11.

A. Чисті функції предметної області (core.py)

Ядро програми не містить глобальних змінних, операцій введення-виведення (print) та не змінює вхідні структури даних.

- **calculate_extra_minutes_cost(record: UsageRecord) -> float**

    - *Опис:* Чиста функція, яка розраховує вартість понаднормових хвилин розмов. Порівнює фактично використані хвилини (minutes_used) з лімітом тарифного плану (free_minutes) і множить різницю на ціну хвилини (price_per_min).
- **calculate_extra_sms_cost(record: UsageRecord) -> float**
    - *Опис:* Чиста функція для обчислення вартості понаднормових SMS-повідомлень на основі перевищення безкоштовного ліміту (free_sms) та вартості одного SMS (price_per_sms).
- **calculate_extra_gb_cost(record: UsageRecord) -> float**
    - *Опис:* Чиста функція, що визначає вартість понаднормового інтернет-трафіку (в гігабайтах) понад ліміт free_gb за тарифною ціною price_per_gb.
- **compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]**
    - *Опис:* Універсальна чиста функція вищого порядку для математичної композиції двох функцій:$(f \circ g)(x) = f\big(g(x)\big)$.
- **make_loyalty_bonus_policy(discount_percent: float) -> BonusFn**
    - *Опис:* Фабрика функцій (функція вищого порядку), яка приймає відсоток знижки та повертає налаштовану функцію-політику BonusFn для розрахунку підсумкової суми зі знижкою лояльності.
- **with_monthly_cost(record: UsageRecord, extra_min: float, extra_sms: float, extra_gb: float, final_cost: float) -> ProcessedUsageRecord**
    - *Опис:* Чиста функція забезпечення іммутабельності. Не мутує вхідний словник record, а створює та повертає **новий словник** із доданими розрахованими полями понаднормових витрат та підсумковим полем monthly_cost.
- **make_tariff_processor(*, accept: FilterFn, bonus_policy: BonusFn) -> Callable[[Iterable[UsageRecord]], dict[str, object]]**

- *Опис:* Параметризована фабрика обробника тарифів (функція вищого порядку). Приймає через typing.Callable правила фільтрації (accept) та політику нарахування бонусів (bonus_policy), повертаючи головну функцію обробки списку записів.

Б. Оболонка побічних ефектів (app.py)

Усі операції взаємодії з користувачем та виведення даних винесені за межі чистих функцій.

- **render_report(result: dict[str, object]) -> None**
  - *Опис:* Функція виведення (I/O). Здійснює друк у консоль (print) результатів опрацювання тарифів,деталізації нарахувань по кожному користувачу та загального доходу (Revenue).
- **run() -> None**
  - *Опис:* Головна функція програми (точка входу). Ініціалізує вхідні дані (sample_records), конфігурує політики через typing.Callable (фільтр активності та знижку), викликає чистий процесор і передає отримані дані у render_report.

3. Структура проєкту

Проєкт організовано відповідно до вимог модульної структури:

- core.py — чисте ядро (розрахунки, типізація TypedDict, Callable-політики).
- app.py — оболонка побічних ефектів (I/O, звітність).
- tests/test_core.py — модульні тести (pytest) на референтну прозорість, відсутність мутацій та правильність функцій.
- README.txt (або README.md) — опис проєкту та інструкція запуску.

4. Висновки

Під час виконання лабораторної роботи було успішно освоєно принципи функціонального програмування на мові Python. На прикладі Варіанта 11 («Мобільний тариф») створено чисте функціональне ядро без побічних ефектів,забезпечено повну референтну прозорість та відсутність мутацій вхідних даних. За допомогою typing.Callable та функцій вищого порядку реалізовано гнучку параметризацію бізнес-логіки (правила тарифікації та бонусів), а будь-які операції I/O ізольовано в окремому модулі оболонки. Написані модульні pytest-тести підтвердили коректність та надійність розробленого рішення.